44

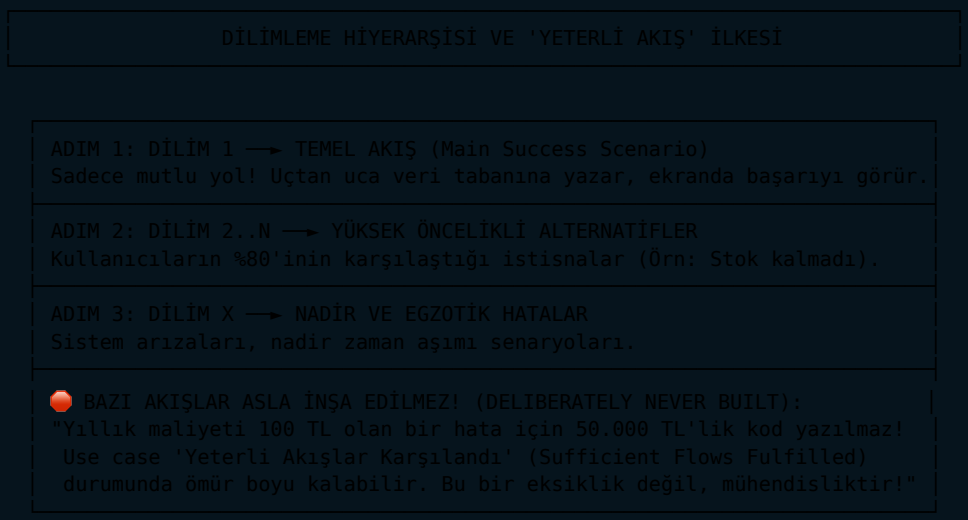
BÖLÜM: DILIMLER (SLICES), TESTLER VE İTERASYON DÖNGÜSÜ | MODULE: SLICES, TESTS, AND THE ITERATION LOOP

Dilimleme Hiyerarşisi: Önce Temel Akış, Sonra Alternatifler (Bazıları Asla Yapılmaz!)

The Slicing Order: Basic Flow First, Alternatives by Priority, Some Never Built

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Yeni açılan bir kahve dükkânını düşünün: İlk gün sadece sıcak kahve satarlar (Temel akış). İkinci hafta vanilyalı ve karamelli şuruplar gelir (Öncelikli alternatifler). 'Organik Madagaskar vanilyalı yulaflı latte'yi ise yılda bir kişi sorar diye menüye hiç koymazlar (Asla inşa edilmeyen akış!). Her şeyi aynı anda yapmaya çalışmak dükkânı batırır!

🇬🇧 ENGLISH (ELI5)

Consider opening a new coffee shop: On Day 1, they only brew classic hot coffee (Basic Flow). By Week 2, they introduce caramel and hazelnut syrups (High-priority alternatives). But 'Organic Madagascar Vanilla Bean Oat Milk Cappuccino' is requested once a year, so they never add it to the menu (Deliberately never built!). Slicing prevents capital exhaustion.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Dilimleme Sırası ve 'Yeterli Akış' Felsefesi: Use-Case 3.0 ve Martinelli'nin metodolojisinde dilimleme sırası katı bir mantığa bağlıdır: 1) Dilim 1 Temel Akış: Sistemin en önemli use case'inin temel akışı, tüm mimari omurgayı (UI, servis, DB, güvenlik) doğrulamak için yeterlidir. 2) Sonraki Dilimler: En sık rastlanan ve iş değeri en yüksek alternatif akışlar sırayla dilimlenir. 3) 'Bazı Akışlar Asla İnşa Edilmez': Geleneksel waterfall zihniyeti şartnamedeki her satırın kodlanmasını şart koşar. Oysa Use-Case 3.0'ın en gerçekçi durumu 'Yeterli Akışlar Karşılandı' (Sufficient Flows Fulfilled) durumudur. Sistem ticari olarak hedefine ulaşmışsa, 5 yılda bir olabilecek çok nadir bir istisna için kod yazmak ve bakım maliyeti üstlenmek israftır. O akış şartnamede analiz edilmiş olarak bekler; manuel operasyonel süreçle idare edilir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Slicing Order and the 'Sufficient Flows Fulfilled' State: Slicing follows an immutable cadence derived from the additive spec architecture: 1) Slice 1 is strictly the Basic Flow. Implementing the happy path end-to-end verifies the architectural backbone (REST endpoints, persistence, transaction boundaries, UI state) and yields early stakeholder validation. 2) Successive Slices incorporate alternative flows sorted strictly by business priority and risk mitigation. 3) Deliberately Unbuilt Flows: A vital tenet of Use-Case 3.0 is that not every analyzed flow must be implemented. A use case can intentionally reside in 'Sufficient Flows Fulfilled' indefinitely. Engineering rare edge-case automation whose cost exceeds manual operational fallback is anti-pattern waste. The specification documents the path; the business consciously opts not to cut the slice.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Dilimlerin Öncelik Sırasıyla Eklemeli (Additive) Olarak Büyütülmesi
package com.maang.aiup.slices;

public class SlicingEvolutionDemo {
    // Aşama 1: Sadece Temel Akış (Slice 1)
    public record BorrowRequest(String bookId, int availableCopies, int currentLoans) {}

    public static class BorrowFlowHandler {
        // Slice 1: Temel Başarı
        public String executeSlice1(BorrowRequest req) {
            return "✅ Kitap ödünç verildi.";
        }

        // Slice 2 Eklenir: A1 (Kopya Yok) ve A2 (Limit Doldu)
        public String executeSlice2(BorrowRequest req) {
            if (req.availableCopies() <= 0) {
                return "⚠ Alternatif A1: Kopya yok, rezervasyona yönlendirildi.";
            }
            if (req.currentLoans() >= 5) {
                return "❌ Alternatif A2: Limit dolu (5 kitap), işlem engellendi.";
            }
            return executeSlice1(req);
        }
    }

    void main() {
        var handler = new BorrowFlowHandler();
        var normalReq = new BorrowRequest("BK-1", 3, 2);
        var noStockReq = new BorrowRequest("BK-2", 0, 1);

        System.out.println("1. İstek: " + handler.executeSlice2(normalReq));
        System.out.println("2. İstek: " + handler.executeSlice2(noStockReq));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Temel akışı canlıya almadan veya paydaşa onaylatmadan alternatif akışları kodlamaya başlamayın; temel akışta yapılacak bir UI değişikliği tüm alternatifleri çöp edebilir.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never begin coding alternative flows before the basic flow is reviewed by stakeholders; a fundamental change in happy path UI invalidates all downstream branches.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Dilimleme sırası sabittir: Önce temel akış, sonra en sık rastlanan alternatifler.
- Her alternatif akışı kodlamak zorunda değilsiniz; bazıları operasyonel olarak çözülür.
- 'Yeterli Akışlar Karşılandı' durumu bir use case için tamamen geçerli bir bitiş durumudur.

🎯 Key Architectural Takeaways

- The slicing order is fixed: basic flow first, followed by frequent alternatives.
- Not all alternative flows must be coded; manual workarounds are often economically sound.
- 'Sufficient Flows Fulfilled' is a valid, mature end-state for a production use case.